PYTHON PROGRAMMING
10 CASE STUDIES

Case Studies 21–30

Simple explanations, algorithms, flowcharts, pseudocode, Python source code, test cases, and conclusions.

Prepared for classroom learning

How to Read the Flowcharts

Each flowchart follows the usual order: Start → input → processing/checks → output → Stop. The diamond-shaped step represents a decision/check. Rules, rates, and sample records are assumed for these exercises; change them if your teacher gives different values.

21. Traffic Violation Penalty Calculator

Problem Statement

Calculate fines for several traffic offences using vehicle type, offence type, and whether the offence is repeated. Show the total and flag disqualification conditions.

Objectives

1. Apply a fixed fine table to each offence.
2. Add the repeat-offence increase when applicable.
3. Flag a driver when a serious offence meets the disqualification rule.

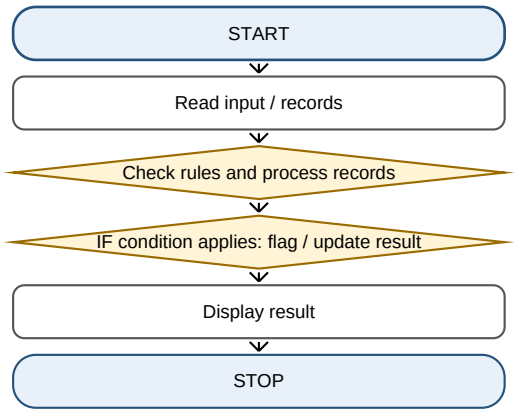
Assumptions

1. Sample rules only: helmet = ■500; speeding = ■1,000; signal jump = ■1,500.
2. A repeat offence doubles its fine.
3. A serious offence (speeding or signal jump) repeated is flagged for review.

Algorithm

1. Read each violation record.
2. Find its base fine; use zero for an unknown offence.
3. Double the fine for a repeat offence.
4. Add it to the total and check the review condition.
5. Display each fine, total, and review flag.

Flowchart



Pseudocode

```
START
SET total = 0
FOR each violation
  fine = base_fine[offence] or 0
  IF repeat THEN fine = fine * 2
  total = total + fine
  IF repeat AND offence is serious THEN review = True
END FOR
PRINT total, review
STOP
```

Python Source Code

```
BASE_FINE = {'helmet': 500, 'speeding': 1000, 'signal': 1500}

def calculate(records):
    total, review, details = 0, False, []
    for vehicle, offence, repeat in records:
        fine = BASE_FINE.get(offence.lower(), 0)
        if repeat:
            fine *= 2
        total += fine
```

```
        if repeat and offence.lower() in ('speeding', 'signal'):
            review = True
        details.append((vehicle, offence, fine))
    return details, total, review

records = [('car', 'speeding', True), ('bike', 'helmet', False)]
details, total, review = calculate(records)
print('Details:', details)
print('Total fine: Rs.', total)
print('Disqualification review:', review)
```

Test Cases

Input / condition	Expected result
speeding, repeat	■2,000; review=True
helmet, not repeat	■500; review=False
unknown offence	■0

Conclusion

The program totals fines consistently and identifies cases that need a disqualification review. The rates are illustrative, not legal advice.

22. Laboratory Chemical Inventory Validation

Problem Statement

Check a sample chemical inventory for storage-category limits and incompatible chemicals stored together.

Objectives

1. Check each quantity against its category limit.
2. Find incompatible pairs in the same storage area.
3. Report any record that needs correction.

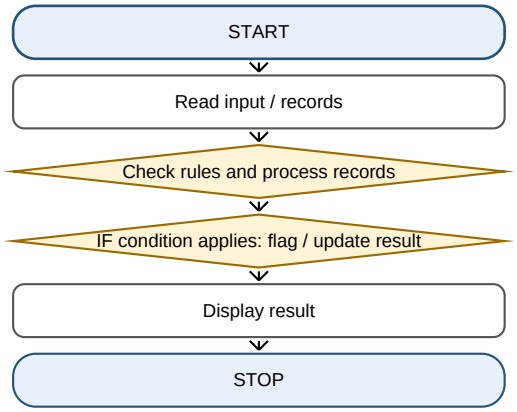
Assumptions

1. Use fictional categories A, B, and C with limits 10, 20, and 15 units.
2. A and B are incompatible when stored together.
3. Each item has a category, quantity, and storage location.

Algorithm

1. Read inventory records.
2. Flag quantities above the category limit.
3. Compare records sharing a location.
4. Flag an incompatible category pair.
5. Print all flags.

Flowchart



Pseudocode

```
START
READ inventory
FOR each item
  IF quantity > category_limit THEN flag quantity
FOR each pair of items
  IF same location AND categories incompatible THEN flag pair
PRINT flags
STOP
```

Python Source Code

```
LIMITS = {'A': 10, 'B': 20, 'C': 15}
INCOMPATIBLE = {('A', 'B'), ('B', 'A')}

def validate(items):
    flags = []
    for item in items:
        if item['qty'] > LIMITS[item['category']]:
            flags.append(f\"{item['name']}: quantity over limit\")
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            a, b = items[i], items[j]
            pair = (a['category'], b['category'])
            if a['location'] == b['location'] and pair in INCOMPATIBLE:
                flags.append(f\"{a['name']} and {b['name']}: incompatible storage\")
    return flags

stock = [
    {'name':'Solvent','category':'A','qty':12,'location':'R1'},
    {'name':'Reagent','category':'B','qty':5,'location':'R1'}]
print(validate(stock))
```

Test Cases

Input / condition	Expected result
A quantity 12 (limit 10)	Quantity flag
A and B in R1	Compatibility flag
A and B in separate rooms	No compatibility flag

Conclusion

The checker catches quantity and storage conflicts in the sample records. Real laboratories must follow their official safety data and procedures.

23. Student Timetable Conflict Detection

Problem Statement

Check a proposed timetable for overlapping class periods and rooms that are unavailable.

Objectives

1. Detect two courses assigned at the same time.
2. Check whether a room is unavailable.
3. Accept the timetable only when no conflicts are found.

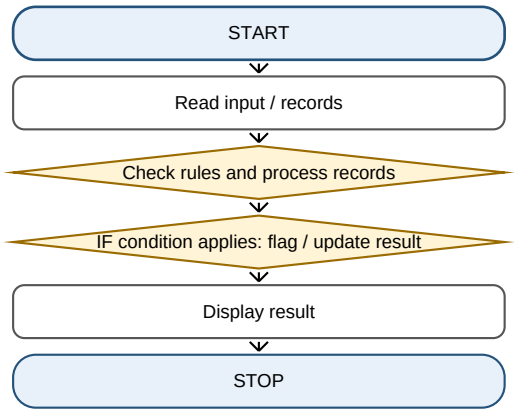
Assumptions

1. A period is represented by start and end minutes.
2. Intervals overlap when start1 < end2 and start2 < end1.
3. Unavailable rooms are listed in a set.

Algorithm

1. Read course records and unavailable rooms.
2. Check every pair for time overlap.
3. Check each room against the unavailable list.
4. Collect conflicts and report validity.

Flowchart



Pseudocode

```
START
READ schedule, unavailable_rooms
FOR each course: check room availability
FOR each pair: IF times overlap THEN report conflict
IF no conflicts THEN print valid
STOP
```

Python Source Code

```
def validate(schedule, unavailable_rooms):
    issues = []
    for c in schedule:
        if c['room'] in unavailable_rooms:
            issues.append(f\"{c['course']}: room unavailable\")
    for i in range(len(schedule)):
        for j in range(i + 1, len(schedule)):
            a, b = schedule[i], schedule[j]
            overlap = a['start'] < b['end'] and b['start'] < a['end']
            if overlap:
                issues.append(f\"{a['course']} overlaps {b['course']}\")
    return issues

classes = [
 {'course':'Python','start':540,'end':600,'room':'R1'},
 {'course':'Maths','start':570,'end':630,'room':'R2'}]
issues = validate(classes, {'R3'})
print('Valid timetable' if not issues else issues)
```

Test Cases

Input / condition	Expected result
09:00–10:00 and 09:30–10:30	Overlap detected
10:00–11:00 and 11:00–12:00	No overlap
Room R3 unavailable	Room flag

Conclusion

The timetable is accepted only when class periods and room assignments meet the stated rules.

24. Courier Charge Optimization

Problem Statement

Calculate delivery costs using weight, dimensions, distance zone, urgency, and restrictions, then choose the cheapest eligible service.

Objectives

1. Estimate charges for available services.
2. Reject options that do not meet parcel restrictions.
3. Choose the lowest-cost eligible option.

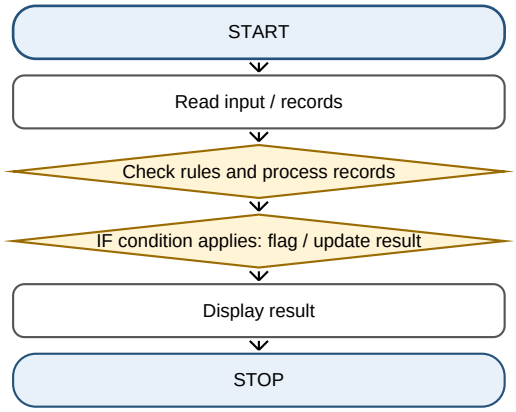
Assumptions

1. Charge = base + weight rate × weight + zone fee + urgency fee.
2. Oversize parcels are not accepted by the sample express service.
3. All amounts and thresholds are examples.

Algorithm

1. Read parcel details and service options.
2. For each service, check its restrictions.
3. Calculate the charge for eligible services.
4. Choose the minimum charge or report no option.

Flowchart



Pseudocode

```
START
READ parcel, services
eligible = empty list
FOR each service
  IF parcel meets restrictions THEN calculate charge and add
IF eligible is empty THEN report none
ELSE choose minimum charge
STOP
```

Python Source Code

```
def choose_service(weight, length, width, height, zone, urgent, services):
    options = []
    volume = length * width * height
    for s in services:
        if volume > s['max_volume'] or weight > s['max_weight']:
            continue
        fee = s['base'] + weight * s['per_kg'] + s['zone_fee'].get(zone, 0)
        if urgent:
            fee += s['urgent_fee']
        options.append((fee, s['name']))
    return min(options) if options else None

services = [
 {'name':'Standard','base':40,'per_kg':20,'zone_fee':{'A':10,'B':25},'urgent_fee':30,'max_volume':50000,'max_weight':20},
 {'name':'Express','base':70,'per_kg':30,'zone_fee':{'A':15,'B':35},'urgent_fee':50,'max_volume':20000,'max_weight':10}]
print(choose_service(2, 20, 15, 10, 'A', True, services))
```

Test Cases

Input / condition	Expected result
2 kg, zone A, urgent	Cheapest eligible service returned
Volume above all limits	None returned
Weight above express limit	Express excluded

Conclusion

The program compares only services that can carry the parcel and returns the lowest sample charge.

25. Progressive Water Tariff Analysis

Problem Statement

Calculate water bills for households using consumption slabs, fixed fees, and penalties, then identify unusually high usage.

Objectives

1. Calculate slab-wise consumption charges.
2. Add fixed fees and any penalty.
3. Compare household usage and flag excessive consumption.

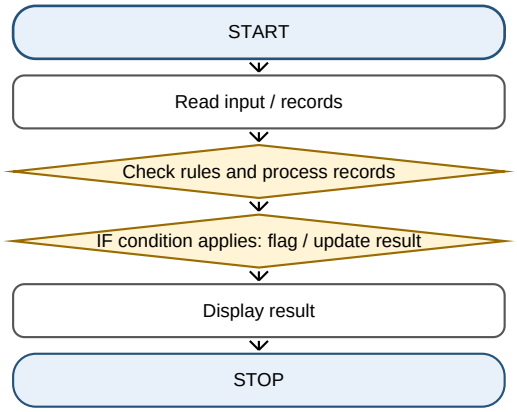
Assumptions

1. First 10 units cost ■5/unit; next 10 cost ■8/unit; above 20 cost ■12/unit.
2. Fixed fee is ■30; usage above 30 units adds a ■100 penalty.
3. More than 40 units is flagged as excessive.

Algorithm

1. Read each household's usage.
2. Calculate charges across the three slabs.
3. Add fixed fee and applicable penalty.
4. Flag usage above 40 units.
5. Print bill and flag.

Flowchart



Pseudocode

```
START
FOR each household
  calculate slab charges
  bill = slab charges + fixed fee
  IF usage > 30 THEN bill += penalty
  IF usage > 40 THEN flag excessive
  PRINT bill and flag
STOP
```

Python Source Code

```
def water_bill(units):
    charge = min(units, 10) * 5
    charge += max(0, min(units - 10, 10)) * 8
    charge += max(0, units - 20) * 12
    bill = charge + 30
    if units > 30:
        bill += 100
    return bill, units > 40

households = {'H1': 8, 'H2': 25, 'H3': 45}
for name, units in households.items():
    bill, excessive = water_bill(units)
    print(name, units, 'units; bill Rs.', bill, 'excessive:', excessive)
```

Test Cases

Input / condition	Expected result
8 units	■70; not excessive
25 units	■220; not excessive
45 units	■570; excessive

Conclusion

The slab method charges different rates for different usage ranges and highlights unusually high consumption.

26. Nested Loop Examination Seating Generator

Problem Statement

Create a classroom seating chart using row and column labels, reserved seats, and alternate-seat restrictions.

Objectives

1. Generate seats within classroom capacity.
2. Mark reserved seats clearly.
3. Leave alternate seats empty when the restriction is enabled.

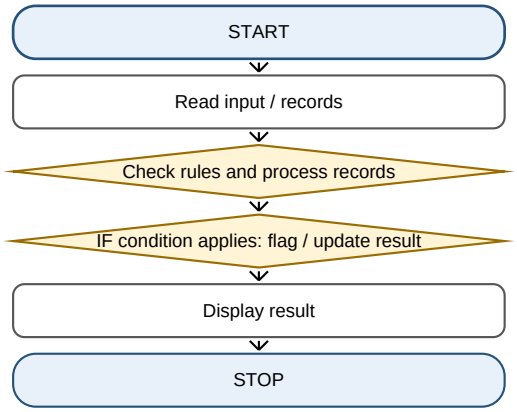
Assumptions

1. Rows and columns are positive integers.
2. Reserved positions are supplied as (row, column) pairs, using zero-based indexes.
3. When alternate seating is enabled, seats with even row+column are left empty unless reserved.

Algorithm

1. Read row and column counts and reserved positions.
2. Use nested loops to visit every seat.
3. Mark reserved seats first.
4. Otherwise mark restricted seats empty or assign a seat number.
5. Print the chart.

Flowchart



Pseudocode

```
START
READ rows, columns, reserved, alternate
FOR row from 0 to rows-1
  FOR column from 0 to columns-1
    IF reserved THEN mark RESERVED
    ELSE IF alternate rule applies THEN mark EMPTY
    ELSE assign seat number
PRINT chart
STOP
```

Python Source Code

```
def seating(rows, cols, reserved, alternate=True):
    chart, number = [], 1
    for r in range(rows):
        line = []
        for c in range(cols):
            if (r, c) in reserved:
                seat = 'RES'
            elif alternate and (r + c) % 2 == 0:
                seat = '--'
            else:
                seat = str(number)
                number += 1
            line.append(seat)
        chart.append(line)
    return chart

for row in seating(3, 4, {(0, 0), (2, 3)}):
    print(' '.join(f'{seat:>3}' for seat in row))
```

Test Cases

Input / condition	Expected result
3 rows × 4 columns	12 positions displayed
Reserved (0,0)	RES shown
Alternate=True	Restricted positions show --

Conclusion

Nested loops visit each classroom position once and produce a readable seating chart.

27. Advanced Numeric Pattern Generator

Problem Statement

Print a hollow diamond pattern with a user-selected size and reject invalid dimensions.

Objectives

1. Validate the requested size.
2. Print a symmetric diamond outline.
3. Keep the pattern aligned using spaces.

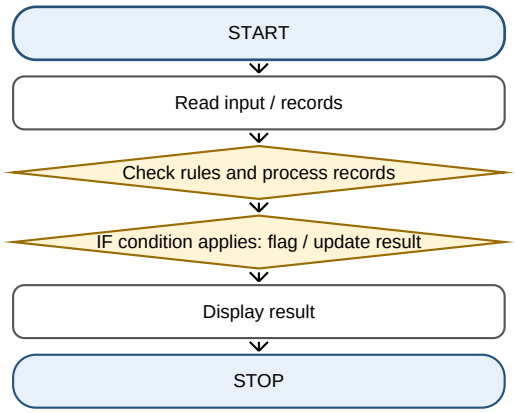
Assumptions

1. The size must be an integer from 1 through 20.
2. The pattern uses stars and spaces.
3. The top half includes the middle row; the bottom half mirrors it.

Algorithm

1. Read size and validate it.
2. For each row in the top half, calculate leading and inner spaces.
3. Print the hollow row.
4. Repeat rows in reverse order for the bottom half.

Flowchart



Pseudocode

```
START
READ n
IF n < 1 OR n > 20 THEN print invalid and STOP
FOR i = 1 TO n
  print hollow row for i
FOR i = n-1 DOWN TO 1
  print hollow row for i
STOP
```

Python Source Code

```
def diamond(n):
    if not isinstance(n, int) or not 1 <= n <= 20:
        return 'Invalid size (choose 1 to 20)'
    rows = []
    for i in list(range(1, n + 1)) + list(range(n - 1, 0, -1)):
        outer = n - i
        if i == 1:
            row = ' ' * outer + '*'
        else:
            row = ' ' * outer + '*' + ' ' * (2 * i - 3) + '*'
        rows.append(row)
    return '\
'.join(rows)

print(diamond(4))
```

Test Cases

Input / condition	Expected result
n=1	One star

Input / condition	Expected result
n=4	7 symmetric rows
n=0 or 21	Invalid size message

Conclusion

The pattern is formed by matching top and bottom rows, creating a symmetric hollow diamond.

28. Repeated Transaction Detection

Problem Statement

Inspect transaction IDs and amounts, find duplicate IDs, flag amounts above a limit, and list suspicious entries.

Objectives

1. Identify repeated transaction identifiers.
2. Flag unusually large amounts.
3. Produce a combined suspicious transaction list.

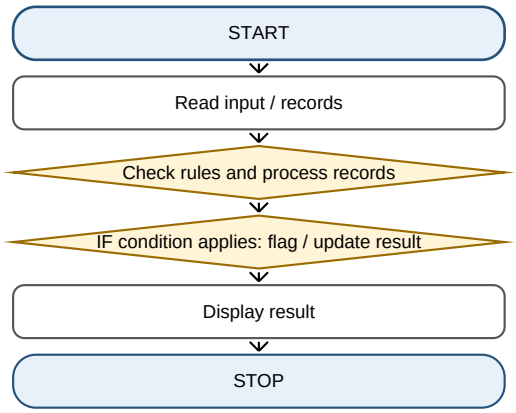
Assumptions

1. Transaction IDs should be unique.
2. An amount greater than ■10,000 is above the sample threshold.
3. A transaction can be flagged for either or both reasons.

Algorithm

1. Read transaction records.
2. Count how often each ID occurs.
3. For each record, check duplicate status and amount threshold.
4. Add suspicious records to the output list.

Flowchart



Pseudocode

```
START
READ transactions
COUNT each transaction ID
FOR each transaction
  IF ID count > 1 OR amount > threshold THEN add to suspicious
PRINT suspicious
STOP
```

Python Source Code

```
from collections import Counter

def find_suspicious(transactions, threshold=10000):
    counts = Counter(t['id'] for t in transactions)
    result = []
    for t in transactions:
        reasons = []
        if counts[t['id']] > 1:
            reasons.append('duplicate ID')
        if t['amount'] > threshold:
            reasons.append('amount over limit')
        if reasons:
            result.append((t, reasons))
    return result

data = [
 {'id':'T1','amount':500},
 {'id':'T2','amount':12000},
 {'id':'T1','amount':700}]
print(find_suspicious(data))
```

Test Cases

Input / condition	Expected result
Same ID twice	Both entries flagged duplicate
■12,000 amount	Amount flag
Unique ID and ■500	Not flagged

Conclusion

The program checks each record for duplicate IDs and high amounts, then reports the reason for each flag.

29. Password Policy Validation Engine

Problem Statement

Evaluate sample passwords using length, character-type, repetition, and prohibited-word rules and explain failures.

Objectives

1. Check minimum length and character categories.
2. Detect repeated adjacent characters.
3. Reject a prohibited substring.
4. Return clear feedback for each failed rule.

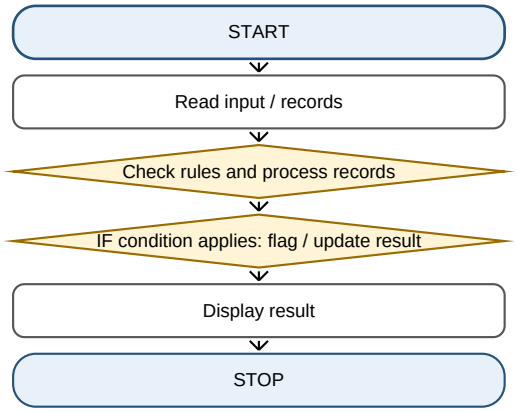
Assumptions

1. Minimum length is 8 characters.
2. A valid password includes uppercase, lowercase, and a digit.
3. Three identical characters in a row are prohibited.
4. The substring 'password' is prohibited, ignoring case.

Algorithm

1. Read a password.
2. Check each policy separately.
3. Store a message for every failed condition.
4. Report valid only if no failures exist.

Flowchart



Pseudocode

```
START
READ password
failures = empty list
CHECK length, uppercase, lowercase, digit
CHECK three repeated characters
CHECK prohibited substring
IF failures empty THEN valid ELSE print failures
STOP
```

Python Source Code

```
import re

def check_password(p):
    errors = []
    if len(p) < 8: errors.append('Use at least 8 characters')
    if not re.search(r'[A-Z]', p): errors.append('Add an uppercase letter')
    if not re.search(r'[a-z]', p): errors.append('Add a lowercase letter')
    if not re.search(r'\\d', p): errors.append('Add a digit')
    if re.search(r'(.)\\1\\1', p): errors.append('Avoid 3 repeated characters')
    if 'password' in p.lower(): errors.append('Avoid the word password')
    return errors or ['Password meets the sample rules']

print(check_password('Hello123'))
print(check_password('password111'))
```

Test Cases

Input / condition	Expected result
Hello123	Meets sample rules
abc	Length/category failures
AaaPassword1	Prohibited word and repetition may be flagged

Conclusion

The validator explains which sample rules fail instead of giving only a pass or fail result.

30. Student Email and Identifier Validation

Problem Statement

Check student email-like strings and registration IDs for required parts, separators, and allowed characters.

Objectives

1. Validate the basic email format.
2. Check the registration ID pattern.
3. Explain malformed or missing parts.

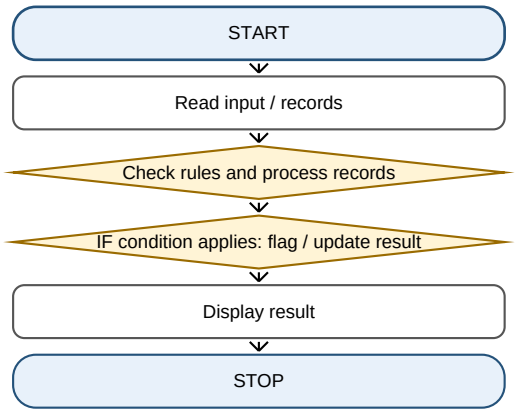
Assumptions

1. Email format is name@college.example; one @ is required and a dot must follow it.
2. Registration ID format is 2 uppercase letters, 4 digits, then 2 uppercase letters (example: CS1234AB).
3. This is a classroom format check, not proof that an email account exists.

Algorithm

1. Read email and registration ID.
2. Check email has one @, non-empty name/domain, and a dot in domain.
3. Check registration ID against the required pattern.
4. Print valid/invalid status and reasons.

Flowchart



Pseudocode

```
START
READ email, reg_id
CHECK email separators and required components
CHECK reg_id matches 2 letters + 4 digits + 2 letters
PRINT status and reasons
STOP
```

Python Source Code

```
import re

def validate_student(email, reg_id):
    errors = []
    if email.count('@') != 1:
        errors.append('Email must contain exactly one @')
    else:
        name, domain = email.split('@')
        if not name: errors.append('Email name is missing')
        if '.' not in domain or domain.startswith('.') or domain.endswith('.'):
            errors.append('Email domain needs a valid dot section')
    if not re.fullmatch(r'[A-Z]{2}\\d{4}[A-Z]{2}', reg_id):
        errors.append('ID must look like CS1234AB')
    return errors or ['Email and ID format are valid']

print(validate_student('sam@college.edu', 'CS1234AB'))
print(validate_student('sam@@college', 'cs12AB'))
```

Test Cases

Input / condition	Expected result
sam@college.edu, CS1234AB	Both formats valid

Input / condition	Expected result
sam@@college, cs12AB	Email and ID errors
@college.edu	Missing email name

Conclusion

The checks catch common formatting mistakes and give students a clear reason to correct their entries.